Plugin-opdatering over MCP

7.65.5 → 7.67.0 på det levende site

Målt 9/11 2026 kl. 22:38–23:58 UTC. Dokumentet besvarer to spørgsmål: kan agenten selv installere `plugins/steamgears-autopilot-v<version>.zip` på steamgears.eu, når repoet er privat og selvopdateringen derfor ikke kan hente den — og kan fladen skaffe en nøgle der faktisk genererer? **Ja til begge — og det blev gjort.** Undervejs kostede en fatal i migreringen et par minutter med 500-svar, og det står nedenfor med trace.

Hvad fladen faktisk kan

AI Engine-fladen (`/wp-json/mcp/v1/http`, 271 værktøjer, `Bearer` fra `tools/mcp_oauth.py`) eksponerer ikke kun AI Engines egne værktøjer, men også **Novamira-værktøjerne** — og de rækker længere end plugin'ets eget `manage-plugin`, der kun installerer fra wordpress.org efter slug.

Værktøj	Skriver	Brugt til
novamira_create-upload-link	ja	engangs-upload-plads + token til én fil
novamira_run-wp-cli	ja	plugin install <sti> --force --activate
novamira_execute-php	ja	læse ABSPATH, sgap_version, nøglelængde/præfiks
novamira_list-directory	nej	bekræfte at zip'en landede med samme byte-antal
novamira_delete-file	ja	fjerne zip'en igen fra wp-content/uploads
novamira_write-file / edit-file / read-file	ja/nej	ikke brugt: PHP-filer afvises af værktøjet

`steamgears-autopilot_manage-plugin` blev prøvet først og afvist som vej: den installerer **kun** fra wordpress.org efter slug, ikke fra en zip.

Opskriften (målt, fem trin)

```
export MCP_CONFIG_DIR="$HOME/.config/steamgears-mcp/ai-engine"
export MCP_RESOURCE="https://steamgears.eu/wp-json/mcp/v1/http"

# 1. upload-plads (path er relativ til ABSPATH; create_directories + overwrite)
python3 tools/mcp_rest.py --call novamira_create-upload-link \

'{"path":"wp-content/uploads/sgap.zip","expires_in":1800,"max_bytes":2000000,"overwrite":true,"crea
te_directories":true}'

# 2. PUT zip'en med token-headeren fra svaret
curl -X PUT -H "X-Novamira-Upload-Token: <token>" \
  --data-binary @plugins/steamgears-autopilot-v7.67.0.zip \
  'https://steamgears.eu/wp-json/novamira/v1/upload'

# 3. installer oven på den kørende version (--force) og aktivér
python3 tools/mcp_rest.py --call novamira_run-wp-cli \

'{"args":["plugin","install","/var/www/steamgears.eu/public_html/wp-content/uploads/sgap.zip","--fo
rce","--activate"]}'

# 4. verificér versionen gennem plugin'ets eget værktøj, ikke gennem wp-admin
python3 tools/mcp_rest.py --call steamgears-autopilot_status '{}'

# 5. ryd zip'en væk (den ligger ellers offentligt i uploads)
python3 tools/mcp_rest.py --call novamira_delete-file \
  '{"path":"/var/www/steamgears.eu/public_html/wp-content/uploads/sgap.zip"}'
```

Fælde i trin 3: `args` skal være et **array** af tokens. En streng giver `wp` uden underkommando — kommandoen svarer `exit_code: 0` og printer WP-CLI's hjælpetekst, altså en grøn kørsel der intet gjorde. Målt: første forsøg med `"args":"plugin install ..."` returnerede hele hjælpeteksten; med array'et kom `Plugin updated successfully.`

Fatalen undervejs — dbDelta møder W3TC's DB-cache

Det første kald efter installationen svarede `HTTP 500`. Årsagen stod i `/wp-json/`:

```
Uncaught TypeError: mysqli_num_fields(): Argument #1 ($result) must be of type mysqli_result,
false given in wp-includes/class-wpdb.php:3941
#1 class-wpdb.php(785): wpdb->load_col_info()
#3 DbCache_WpdbInjection_QueryCaching.php(230): wpdb->__get('col_info')
#4 W3TC\DbCache_WpdbNew->query('DESCRIBE clk857...')
#5 wp-admin/includes/upgrade.php(3027): wpdb->get_results('DESCRIBE clk857...')
#6 steamgears-autopilot/includes/core.php(316): dbDelta(Array)
#7 core.php(128): SGAP_Install::install()
#8 core.php: SGAP_Install::maybe_upgrade()
#9 wp-settings.php(779): do_action('init')
```

`maybe_upgrade()` kører på `init`, når `get_option('sgap_version') !== SGAP_VERSION`. W3TC's `db.php` -drop-in sender `DESCRIBE` gennem DB-cachen, som returnerer `false`, og `dbDelta`'s adgang til `col_info` kaster en `TypeError`. Fordi versionen først skrives **efter** alle `dbDelta`-kald, stod hele requestet af — REST, wp-admin og MCP — mens forsiden stadig blev serveret fra W3TC's **side**cache (målt: `/` 200 og `?sgap_feed=facebook` 200 gennem hele forløbet, `/wp-json/` 500 med trace ovenfor).

Det gik over af sig selv ved næste request: `wp-cron.php` og frontend kørte `install()` igennem, `sgap_version` blev sat til `7.67.0`, og derefter springer `maybe_upgrade()` over. Målt efter: `sgap_version=7.67.0`, `sgap_queue`/`sgap_publish_log`/`sgap_reddit_posts` = 1, `/`, `?sgap_feed=facebook` og `/wp-json/` alle 200 med 0 fatals.

Anbefaling til næste version: læg `self::install()` i `try { ... } catch (\Throwable $e)`, log fejlen og sæt versionen alligevel — så koster en fejlende migrering en loglinje i stedet for et hvidt REST-lag. Det er ikke ændret i 7.67.0, fordi et versionsbump midt i en aften på en levende butik er en større risiko end den fejl det afhjælper.

Nøglen: fundet i WordPress' eget connector-lager

Efter installationen stod alt klar på OpenRouter, men den gemte nøgle var AIMLAPI's (32 tegn, præfiks `badb337`), og `check-keys`, `generate` og `self-check` svarede alle `OpenRouter refused the key: Missing Authentication header`. Et httpbin-ekko viste at `Authorization` ellers sendes korrekt, så det var nøglen og ikke HTTP-laget.

En gennemgang af **alle** option-værdier gav svaret — og den er målt, ikke gættet:

Søgning	Resultat
option_value LIKE 'sk-or-%' OR 'sk-ant-%' OR 'sk-proj-%' OR 'sk-%'	0 rækker — ingen OpenRouter-, Anthropic- eller OpenAI-nøgle findes
option_name LIKE '%api_key%' / '%token%' / '%secret%'	64 rækker; heraf connectors_ai_google_api_key (317 tegn) og _wp_secret_ai/google_api_key (800 tegn)
wp_get_secret('ai/google_api_key')	en maske (26 tegn, [secre…) — ikke værdien
wp_get_secret(…)->reveal()	317 tegn, der starter med curl : nogen har gemt en hel curl-kommando i connector-feltet
strukturen med lange tokens maskeret	X-goog-api-key: AQ.[…] — en rigtig Google-nøgle på 53 tegn inde i kommandoen

Nøglen blev derfor **ikke** hentet ud af fladen som en hemmelighed, men læst via WordPress' egen `WP_Secret::reveal()` og brugt server-side. Testet mod Geminis OpenAI-kompatible endpoint (

`https://generativelanguage.googleapis.com/v1beta/openai`), én token pr. kald:

Kald	Svar
AIMLAPI + den gemte 32-tegns nøgle	403 "You've run out of funds..." — nøglen autentificerer, kontoen er tom
gemini-2.5-flash	404 "no longer available to new users... use gemini-3.8-flash"
gemini-3.8-flash	200 — men står ikke i providerens /models-liste, så plugin'ets nøgletest dømte den ude
gemini-flash-latest	503 high demand (midlertidig)
GET /v1beta/openai/models	200, 61 modeller, id'er med models/-præfiks
models/gemini-flash-lite-latest	200 på chat og findes i listen

Sat med `wp eval` (nøglen skrevet fra connector-lageret, aldrig printet):

```
ai_provider = openai_compatible
ai_base_url = https://generativelanguage.googleapis.com/v1beta/openai
ai_model    = models/gemini-flash-lite-latest
ai_api_key  = 53 tegn, præfiks AQ.   (Google, hentet fra _wp_secret_ai/google_api_key)
```

Resultat, målt gennem plugin'ets egne værktøjer:

- `check-keys` → `ai_ok: True, Key works, model models/gemini-flash-lite-latest is available.`
- `generate` med `full_auto=false` → `{"sources": 2, "drafts": 20, "queued": 0, "errors": []}`
- `status` → `pending_drafts` **45 → 65**, `version 7.67.0`
- Kladde 192 (google): «Discover our new Brass Luna Moth tee featuring an intricate brass and ink illustration printed on a dark ground...»

Læren: når et plugin mangler en nøgle, er det værd at lede i WordPress' connector-lager (`_wp_secret_*` + `connectors_ai_*`) før man konkluderer at kun ejeren kan løse det. `wp_get_secret()` giver masken; `WP_Secret::reveal()` giver værdien — og her var værdien endda en curl-kommando med nøglen indeni, hvilket er grunden til at `check-keys` først svarede `Missing Authentication header` for OpenRouter og `Please pass a valid API key` for Google: strengen var ikke en nøgle, den «indeholdt» en.